

UNIVERSIDAD DEL VALLE DE GUATEMALA

ICC30853020261 - Deep Learning

Sección 31

Javier Fong



Tarea #6 - Transfer Learning

Isabella Obando - 23074

Anthony Lou - 23410

30/09/2026

2. Exploración y preparación de los datos

Cargue el dataset y responda las siguientes preguntas:

- ¿Cuántas observaciones y cuántas clases tiene el dataset? ¿Las clases están balanceadas?

Tiene 60,000 imágenes, de las cuales 50,000 corresponden al conjunto de entrenamiento y 10,000 al conjunto de test. El dataset tiene 10 clases y están balanceadas. Debido a este balance, se espera que las métricas de accuracy y F1 macro sean muy parecidas. Para la evaluación, el conjunto de entrenamiento original se divide en 45,000 imágenes para entrenamiento y 5,000 para validación.

- ¿Cuál es la resolución y el número de canales de las imágenes? Calcule la media y desviación estándar por canal del conjunto de entrenamiento y compártelas con las estadísticas de normalización de ImageNet.

Las imágenes de CIFAR-10 tienen una resolución de 32×32 píxeles y cuentan con 3 canales RGB.

Canal	CIFAR-10	ImageNet
Desviación	0.25	0.23
R	0.49	0.485
G	0.48	0.456
B	0.45	0.406

- Visualice al menos 5 ejemplos por clase. ¿Qué pares de clases esperan que sean más difíciles de distinguir y por qué?

Gato – perro: pueden presentar poses, texturas y tamaños similares.

Automóvil – camión: tienen formas y contextos muy parecidos.

Venado – caballo: ambos son cuadrúpedos y pueden aparecer sobre fondos verdes o cafés.

Avión – barco: pueden aparecer como objetos pequeños sobre fondos similares.

Avión – pájaro: ambos pueden ser objetos pequeños sobre un fondo azul de cielo.

- ¿Qué transformaciones de preprocesamiento necesita cada modelo? Defina un pipeline para la CNN desde cero (resolución nativa) y otro para VGG-16 (redimensionamiento y normalización de ImageNet). ¿Qué pasa con el número de píxeles, y por lo tanto con el cómputo, al pasar de 32×32 a 224×224 ?

Para la CNN desde cero, se mantiene la resolución nativa de CIFAR-10, es decir, 32×32 píxeles. El pipeline de la CNN sería: RandomCrop, luego RandomHorizontalFlip, después ToTensor, y por último Normalize con las estadísticas de CIFAR-10.

Para VGG-16, las imágenes primero se redimensionan a la resolución utilizada por el modelo y luego se normalizan con las estadísticas de ImageNet. El preset oficial de VGG16 utiliza resize 256 + center crop 224, se utiliza un redimensionamiento directo.

Hay aproximadamente 49 veces más píxeles cuando se pasa de 32×32 a 224×224 . Como el costo de las capas convolucionales es aproximadamente proporcional a $\sqrt{H \times W}$, el número de operaciones de convolución también aumenta aproximadamente $49 \times$. Con 128×128 , el aumento respecto a 32×32 es de $16 \times$, por lo que resulta mucho más manejable computacionalmente.

- ¿Qué técnicas de data augmentation utilizarán y por qué? Justifique por qué la augmentation se aplica sólo al conjunto de entrenamiento.

La data augmentation se aplica únicamente al conjunto de entrenamiento porque su objetivo es regularizar el modelo y ampliar artificialmente la distribución de ejemplos que observa durante el aprendizaje. En cambio, validación y test deben representar imágenes reales y mantenerse deterministas y comparables entre modelos. Vamos a usar las siguientes técnicas de data augmentation:

- RandomCrop: permite introducir pequeñas traslaciones en las imágenes y ayuda a que el modelo sea menos sensible a la posición exacta del objeto.
- RandomHorizontalFlip: permite generar variaciones de las imágenes mediante volteos horizontales, aprovechando que los objetos de CIFAR-10 generalmente pueden aparecer orientados hacia la izquierda o hacia la derecha.

Divida el conjunto de entrenamiento original en entrenamiento y validación (por ejemplo, 45,000 / 5,000, estratificado por clase) y reserve el conjunto de test oficial para la evaluación final.

3. Investigación: transfer learning en PyTorch

Investigue las clases, utilidades y herramientas necesarias para reutilizar un modelo preentrenado y para medir su costo computacional. Para cada una, describa brevemente su propósito y sus parámetros más relevantes. Adicionalmente, investigue brevemente los siguientes conceptos: diferencia entre MACs y FLOPs y cómo se relacionan; diferencia entre parámetros totales y parámetros entrenables.

Los modelos `torchvision.models.vgg16` y `VGG16_Weights` construyen la arquitectura y, si se le pasa un enum de pesos, descarga los pesos pre entrenados en ImageNet; con `weights=None` queda inicializada al azar, y `progress` controla la barra de descarga. El enum guarda la URL, metadatos y `weights.transforms()`, que devuelve el preprocesamiento con el que se entrenó: `resize` a 256, `center crop` 224, escalado a `[0,1]` y normalización con media/std de ImageNet.

La estructura `“model.features”` es un `nn.Sequential` con 13 convoluciones 3×3 + ReLU agrupadas en 5 bloques que terminan en `MaxPool2d(2)`, cada bloque divide la resolución entre 2. `“model.avgpool”` es `nn.AdaptiveAvgPool2d((7,7))`: produce siempre un mapa de 7×7 sin importar la entrada, lo que permite usar 128×128 sin cambiar el clasificador. El clasificador concentra 119.6 M de los 134 M parámetros, mientras que casi todos los FLOPs están en features.

Requires_grad y congelamiento.

Cada `nn.Parameter` tiene `requires_grad`, si es `False`, autograd no calcula su gradiente, el optimizador no lo actualiza y no se guardan activaciones para calcularlo. Se congela un bloque con `for p in model.features[24:31].parameters(): p.requires_grad = False` y se descongela poniéndolo en `True`, conviene pasar al optimizador solo los parámetros entrenables. Si todo lo que está debajo de cierta capa está congelado, el backward no llega hasta ahí, lo que ahorra cómputo y memoria.

`Model.train()` vs `model.eval()` y `torch.no_grad()`.

`train()/eval()` solo cambian el comportamiento de capas como Dropout y BatchNorm, no afectan los gradientes. `torch.no_grad()` desactiva la construcción del grafo computacional, así

que no se guardan activaciones y la inferencia es más rápida y usa menos memoria. Para evaluar se usan ambos a la vez.

Grupos de parámetros.

Los optimizadores aceptan una lista de diccionarios, cada grupo puede tener su propio lr, weight_decay, momentum, y lo no especificado toma el valor por defecto. Quedan en optimizer.param_groups y los schedulers escalan el lr de cada grupo por separado, lo que permite usar un lr pequeño en el backbone y uno mayor en la cabeza nueva.

Conteo de parámetros y operaciones.

Usamos torchinfo.summary que lista por capa la forma de salida, parámetros y "mult-adds" totales, y FlopCountAnalysis, cuyo .total() y .by_module() cuentan operaciones por módulo. thop.profile y get_model_complexity_info hacen lo mismo con otras convenciones.

Memoria y tiempo en GPU.

Las operaciones CUDA son asíncronas: la CPU encola kernels y sigue, así que antes de leer el reloj hay que llamar torch.cuda.synchronize() para que el tiempo medido incluya el trabajo real. max_memory_allocated() devuelve el pico de bytes ocupados por tensores desde el inicio o desde reset_peak_memory_stats(); no incluye la caché reservada por el allocator ni el contexto CUDA.

MACs vs FLOPs.

Un MAC es $a \leftarrow a + b \cdot c$, una multiplicación y una suma, por lo que $\text{FLOPs} \approx 2 \times \text{MACs}$. Para una convolución, $\text{MACs} = H_{\text{out}} \cdot W_{\text{out}} \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$. Las herramientas no son consistentes en la convención, así que siempre hay que aclarar cuál se reporta.

6. Discusión y análisis

Responda con base en sus resultados:

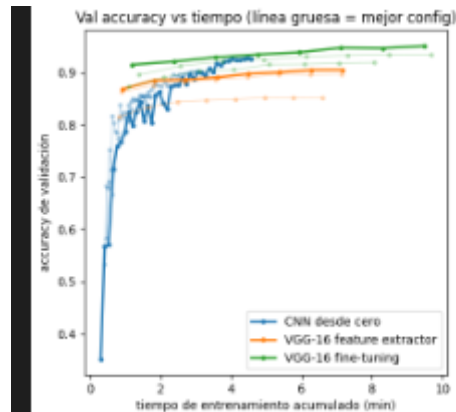
- ¿Cuál de los tres modelos obtuvo el mejor desempeño en test? ¿Qué tan grande fue la diferencia y justifica el costo adicional en parámetros, FLOPs y tiempo de entrenamiento?

	loss	acc	precision	recall	f1
CNN desde cero	0.2590	0.9218	0.9218	0.9218	0.9215
VGG-16 feature extractor	0.4371	0.8974	0.8975	0.8974	0.8974
VGG-16 fine-tuning	0.2327	0.9469	0.9469	0.9469	0.9468

El mejor fue VGG-16 con fine-tuning: F1 de 0.9468, contra 0.9215 de la CNN y 0.8974 del feature extractor. Le sacó +2.5 puntos a la CNN, pero con 28.6× más parámetros, 24.4× más FLOPs de inferencia y 2.1× más tiempo de entrenamiento.

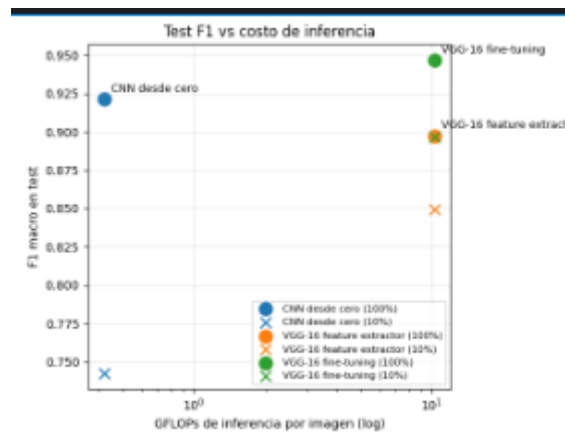
Si lo único que importa es la exactitud y hay GPU, sí lo justifica, porque el tiempo extra de entrenamiento es de apenas 5 minutos. Sí importa la eficiencia, no: 24× más cómputo por 2.5 puntos es caro. El feature extractor no justifica su costo en ningún caso, porque cuesta lo mismo que el FT y rinde menos que la CNN.

- ¿Qué modelo ofrece la mejor relación entre desempeño y recursos? Justifiquen su respuesta usando las gráficas de la sección 5.



La CNN desde cero. En la gráfica de F1 vs FLOPs de inferencia queda sola a la izquierda con F1 de 0.92, mientras los dos VGG necesitan $24\times$ más cómputo. Además tiene el mejor F1 por PFLOP de entrenamiento: 0.41, contra 0.23 del FE y 0.14 del FT.

El FT converge en menos épocas, como se ve en la gráfica de val accuracy vs tiempo. Aun así, cada época le cuesta 71 s contra 7 s de la CNN. El FE queda dominado por la CNN: es más caro y además peor.



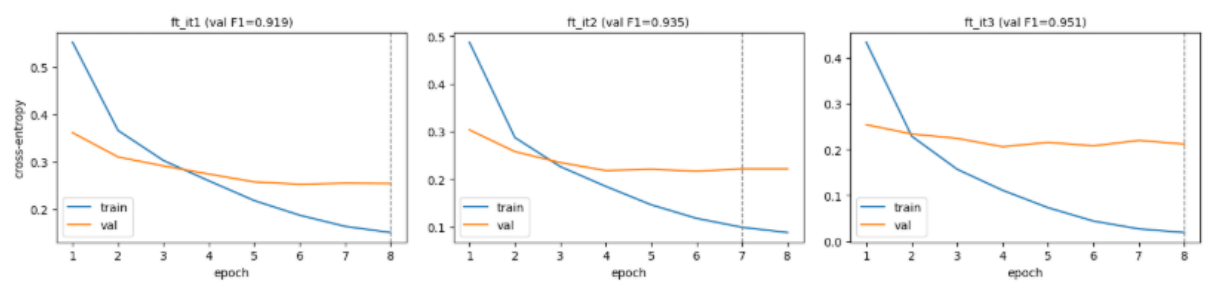
- VGG-16 fue entrenada con fotografías de 224×224 y CIFAR-10 tiene imágenes de 32×32 . ¿Por qué las features preentrenadas siguen siendo útiles? ¿Qué efecto tiene el redimensionamiento en el desempeño y en el costo?

	CIFAR mean	ImageNet mean	CIFAR std	ImageNet std
R	0.4911	0.485	0.2469	0.229
G	0.4821	0.456	0.2434	0.224
B	0.4464	0.406	0.2616	0.225

Las capas de VGG aprenden bordes, texturas y partes de objetos, y eso es bastante genérico. Además, las 10 clases de CIFAR tienen equivalentes en ImageNet y las estadísticas de color son muy parecidas. La prueba es que el feature extractor, sin entrenar ninguna convolución, llega a 0.90 de accuracy.

Redimensionar a 128 no agrega información, pero pone los objetos a una escala parecida a la que vio VGG. También evita que el mapa final quede en 1×1 : con 128 queda en $4 \times 4 \times 512$. El costo es alto, porque son $16 \times$ más píxeles que con 32×32 (con 224 serían $49 \times$). Por eso VGG hace 5.13 GMACs contra 0.21 de la CNN. Ojo: no corrieron una ablación con otras resoluciones, así que el efecto sobre la accuracy es un argumento teórico, no un resultado medido.

- En el fine-tuning, ¿cómo afectó el número de bloques descongelados y el learning rate del backbone? ¿Observaron señales de overfitting o de catastrophic forgetting?



Pasar de 1 a 2 bloques descongelados subió el val F1 de 0.919 a 0.935. Subir el LR del backbone de $1e-5$ a $1e-4$ lo llevó a 0.951. Con $1e-5$ y solo 8 épocas, el backbone casi no alcanzaba a adaptarse.

Hay overfitting leve en ft_it3: la train loss llega a 0.019 y la val loss se queda en ~ 0.21 . Aun así, la val accuracy siguió subiendo. El caso más claro es fe_it1, donde la val loss sube de 0.33 a 0.40 mientras la train loss cae a 0.013. No hubo catastrophic forgetting: con un LR $10 \times$ mayor el modelo mejoró desde la primera época, sin colapsar.

- Con base en el experimento de la sección 4.1, ¿cuál modelo se vio más afectado al reducir los datos? ¿Cómo relacionan esto con lo discutido en clase sobre cuándo usar feature extraction, fine-tuning o entrenar desde cero?

	acc 100%	acc 10%	F1 100%	F1 10%	caída F1 (pts)	caída relativa (%)
modelo						
CNN desde cero	0.9218	0.7425	0.9215	0.7424	17.9078	19.4332
VGG-16 feature extractor	0.8974	0.8495	0.8974	0.8494	4.8019	5.3510
VGG-16 fine-tuning	0.9469	0.8969	0.9468	0.8965	5.0302	5.3125

La CNN fue la más afectada por mucho: perdió 17.9 puntos de F1 (de 0.92 a 0.74). El FE y el FT solo perdieron ~ 5 puntos. Con poco dato, incluso el feature extractor (0.85) le gana con amplitud a la CNN, o sea que el orden entre ellos se invierte.

Esto coincide con lo visto en clase. Con pocos datos y un dominio similar conviene feature extraction o fine-tuning, porque las features ya vienen aprendidas. Entrenar desde cero solo es competitivo cuando hay muchos datos, como con las 45k imágenes.

- Analicen las matrices de confusión: ¿qué pares de clases se confunden más en cada modelo? ¿Los tres modelos cometen los mismos errores?

CNN desde cero — test acc 0.922											
real	airplane	934	3	13	7	6	1	1	1	24	10
	automobile	4	907	0	0	1	0	1	0	4	23
	bird	24	0	974	18	28	18	19	17	1	3
	cat	8	0	19	799	23	108	22	11	6	4
	deer	5	1	19	11	929	12	7	15	2	0
	dog	3	0	7	58	17	696	5	11	1	2
	frog	2	1	10	17	6	5	956	2	0	1
	horse	7	0	2	11	13	19	0	908	1	1
	ship	19	4	2	2	2	0	0	1	919	11
	truck	9	18	1	2	0	1	0	1	9	919
airplane	bird	cat	deer	dog	frog	horse	ship	truck			
automobile	predicción										

VGG-16 feature extractor — test acc 0.897											
real	airplane	929	4	13	4	8	0	3	2	31	9
	automobile	7	905	0	2	0	0	1	1	7	43
	bird	18	1	983	18	43	15	22	11	2	1
	cat	6	1	21	821	28	80	24	14	4	3
	deer	6	1	30	23	896	11	30	29	1	3
	dog	3	3	15	305	17	875	5	17	0	0
	frog	4	2	22	21	19	4	923	2	1	2
	horse	9	0	12	15	28	29	1	913	3	4
	ship	31	9	5	2	1	2	0	1	941	6
	truck	13	47	0	0	0	0	0	1	14	905
airplane	bird	cat	deer	dog	frog	horse	ship	truck			
automobile	predicción										

VGG-16 fine-tuning — test acc 0.947											
real	airplane	962	3	11	1	4	1	1	0	11	6
	automobile	4	966	0	0	0	0	0	0	4	26
	bird	19	0	999	10	18	11	11	2	0	0
	cat	6	1	8	879	14	88	11	8	3	2
	deer	3	1	13	10	951	11	2	8	0	1
	dog	2	0	8	59	16	901	0	11	0	1
	frog	1	0	4	13	5	1	976	0	0	0
	horse	5	0	3	2	15	9	0	966	0	0
	ship	21	7	4	1	0	0	2	0	962	3
	truck	4	23	1	0	0	0	0	0	6	969
airplane	bird	cat	deer	dog	frog	horse	ship	truck			
automobile	predicción										

En los tres modelos el par más confundido es cat-dog (166 / 185 / 127 errores), y cat es la clase con peor recall. También se repiten automobile-truck, airplane-ship y bird-deer. En esencia cometen los mismos errores, en clases con forma o fondo parecidos.

Sí hay diferencias. La CNN confunde cat-frog, lo que sugiere que se guía más por color y textura. El FE tiene el doble de errores en automobile-truck (90) y en vehículos rinde peor que la CNN. El FT reduce todos los pares, y bird-deer cae de 73 a 31 frente al FE.

- Si tuvieran que desplegar un modelo para este problema en (a) un servidor con GPU y (b) un dispositivo móvil, ¿qué modelo elegirían en cada caso y por qué? ¿Qué arquitectura preentrenada alternativa a VGG-16 considerarían para el caso (b)?

(a) Servidor con GPU: VGG-16 con fine-tuning. Es el más preciso, y a 1.5 ms por imagen con batch de 128 pasa de 600 imágenes por segundo en una T4. El tamaño (~537 MB) no es problema en un servidor.

(b) Móvil: de los tres, la CNN. Pesa ~19 MB, hace 0.42 GFLOPs y trabaja en 32×32 nativo con 92 % de accuracy. VGG-16 es inviable en un teléfono. Como alternativa preentrenada recomendaría MobileNetV3 o EfficientNet-B0, que están diseñadas para móvil y conservan la ventaja del transfer learning.

	CNN desde cero	VGG-16 feature extractor	VGG-16 fine-tuning
Params totales (M)	4.6924	134.3015	134.3015
Params entrenables (M)	4.6924	119.5868	132.5660
MACs/img (G)	0.2099	5.1307	5.1307
FLOPs/img Inferencia (G)	0.4198	10.2615	10.2615
Latencia bs=1 (ms/img)	1.1192	4.9096	4.9090
Latencia bs=128 (ms/img)	0.0773	1.5299	1.5291
s/epoch	6.8565	53.7296	71.0829
Epoch mejor val	39.0000	8.0000	8.0000
Epochs corridos	40.0000	8.0000	8.0000
Tiempo hasta mejor (min)	4.4626	7.1639	9.4777
Tiempo total (min)	4.5710	7.1639	9.4777
Memoria pico GPU (GB)	0.8138	3.8139	3.2682
FLOPs entrenamiento/img (G)	1.2593	10.7398	18.5915
Cómputo entrenamiento total (PFLOPs)	2.2668	3.8663	6.6930
Cómputo hasta mejor epoch (PFLOPs)	2.2101	3.8663	6.6930
Test acc	0.9218	0.8974	0.9469
Test precision	0.9218	0.8975	0.9469
Test recall	0.9218	0.8974	0.9469
Test F1	0.9215	0.8974	0.9468
Test acc (10%)	0.7425	0.8495	0.8969
Test F1 (10%)	0.7424	0.8494	0.8965
Hardware: Tesla T4			

git: <https://github.com/Isabella1911/DL-Lab-6-Transfer-Learning-.git>